

RetailPOS

Technical Documentation

Microservices Architecture · .NET 9 · Angular 18

Generated: May 04, 2026
Version: 1.0.0
Status: Internal / Confidential

Project	RetailPOS — Point of Sale System
Architecture	Microservices (Event-Driven)
Backend	.NET 9 / ASP.NET Core Web API
Frontend	Angular 18 (TypeScript)
Messaging	RabbitMQ + MassTransit
Database	Microsoft SQL Server 2025
Auth	JWT + OTP + Google OAuth
Containerization	Docker / Docker Compose

Table of Contents

1. Project Overview

- 1.1 Purpose & Scope
- 1.2 Key Features
- 1.3 Technology Stack

2. System Architecture

- 2.1 High-Level Design
- 2.2 Microservices Overview
- 2.3 Infrastructure Components
- 2.4 Event-Driven Communication

3. Microservices Reference

- 3.1 Identity Service
- 3.2 Catalog Service
- 3.3 Orders Service
- 3.4 Payment Service
- 3.5 Returns Service
- 3.6 Admin Service
- 3.7 Notification Service
- 3.8 AI Service
- 3.9 API Gateway

4. Authentication & Authorization

- 4.1 Authentication Flow
- 4.2 JWT Token Strategy
- 4.3 Role-Based Access Control
- 4.4 Permission System

5. Data Model & Contracts

- 5.1 Domain Models
- 5.2 Message Contracts
- 5.3 Saga Patterns

6. API Reference

- 6.1 Gateway Routes
- 6.2 Auth Endpoints
- 6.3 Catalog Endpoints
- 6.4 Orders Endpoints
- 6.5 Admin Endpoints

7. Frontend Architecture

- 7.1 Angular Application Structure
- 7.2 Feature Modules
- 7.3 Services & State

8. Deployment

8.1 Docker Compose Setup

8.2 Port Allocation

8.3 Environment Variables

9. Testing Strategy

1. Project Overview

1.1 Purpose & Scope

RetailPOS is a cloud-native, multi-tenant Point of Sale system designed to serve multi-store retail operations. Built on a microservices architecture, it decouples every business domain (identity, catalog, orders, payments, returns, administration, notifications, and AI assistance) into independently deployable services that communicate through an asynchronous event bus. The Angular 18 single-page application provides cashiers and administrators with a responsive, real-time interface.

1.2 Key Features

- **Multi-Tenant Store Management**

Each store operates within an isolated tenant boundary; staff are scoped to their assigned store.

- **Real-Time Billing & Checkout**

Cashiers compose carts, apply taxes, and finalize orders through a saga-orchestrated checkout flow.

- **Inventory Management**

Automatic stock deduction on checkout and stock restoration on returns, with manual adjustment support.

- **Payment Processing**

Pluggable payment modes (Cash, Card, UPI) tracked per transaction with full refund capability.

- **Returns & Refunds**

A dedicated return saga orchestrates stock restoration and payment reversal with approval workflow.

- **Admin Dashboard**

KPI cards, sales reports, inventory alerts, and staff management for store managers and admins.

- **AI Chatbot**

Integrated AI assistant powered by Claude for in-app query resolution.

- **Structured Logging**

Centralised log aggregation via Seq with Serilog structured logging across all services.

- **OTP & Google OAuth**

Two-factor login via email OTP with optional Google Sign-In.

1.3 Technology Stack

Layer	Technology	Purpose
Backend Services	.NET 9 / ASP.NET Core	REST APIs for all microservices
Frontend	Angular 18 + TypeScript	Single-Page Application
Message Broker	RabbitMQ 3 + MassTransit	Async event bus & saga coordination
Databases	MS SQL Server 2025	Per-service relational databases
Caching	Redis (Alpine)	Session/token caching
API Gateway	Ocelot	Reverse proxy, routing, circuit-breaking
Logging	Seq + Serilog	Centralised structured logging
Auth	JWT + BCrypt + Google OAuth	Authentication & password hashing
Real-Time	SignalR	Push notifications to UI
Containerization	Docker + Docker Compose	Local & cloud deployment
ORM	Entity Framework Core 9	Database migrations & queries
Testing	xUnit + Moq	Unit & integration tests

2. System Architecture

2.1 High-Level Design

The system follows a microservices architecture with an API Gateway as the single entry point for clients. All inter-service communication uses an asynchronous event bus (RabbitMQ), except for gateway routing which is synchronous HTTP. Each service owns its own database, preventing tight coupling at the data layer.

```
[ Angular 18 SPA - POS UI ]
| HTTPS
[ Ocelot API Gateway :5000 ]

+-----+-----+-----+-----+-----+-----+-----+-----+
Identity Catalog Orders Payment Returns Admin Notif AI
:5001 :5002 :5003 :5005 :5006 :5004 :5176 :5007
All services publish/subscribe via RabbitMQ message bus
[ RabbitMQ Message Broker :5672 ] <--> [ MassTransit Sagas ]
[ MS SQL Server :1433 ] [ Redis :6379 ] [ Seq Logger :8081 ]
```

2.2 Microservices Overview

Service	Port	Database	Key Responsibilities
Identity Service	5001	IdentityDb	Auth, user registration, OTP, JWT, Google OAuth, role management
Catalog Service	5002	CatalogDb	Product CRUD, category management, stock levels, per-store filtering
Orders Service	5003	OrdersDb	Bill/cart creation, checkout saga initiation, order lifecycle
Payment Service	5005	PaymentDb	Payment processing, refund handling, payment status tracking
Returns Service	5006	ReturnsDb	Return initiation, approval workflow, return saga coordination
Admin Service	5004	AdminDb	Dashboard KPIs, reports, inventory adjustments, store & staff mgmt
Notification Service	5176	NotificationDb	SignalR real-time hub, notification persistence, event-triggered alerts
AI Service	5007	—	AI chatbot endpoint via Claude API integration
API Gateway	5000	—	Ocelot reverse proxy, circuit-breaker (3 exceptions / 10s break)

2.3 Infrastructure Components

RabbitMQ (Port 5672 / Management 15672)

Message broker providing durable queues and topic exchanges. MassTransit wraps RabbitMQ to provide consumer registration, retry policies, and saga state persistence.

Microsoft SQL Server 2025 (Port 1433)

Each microservice maintains a dedicated database with schema migrations managed by EF Core. A single SQL Server instance is used in Docker Compose; in production each service would use isolated instances.

Redis (Port 6379)

In-memory key-value store used for caching tokens, session data, and OTP codes with configurable TTL.

Seq (Port 8081 / Ingestion 5341)

Structured log server. All services ship Serilog events over HTTP to Seq, enabling centralized filtering, alerting, and dashboards without third-party SaaS.

2.4 Event-Driven Communication

Services communicate exclusively via domain events defined in the shared RetailPOS.Contracts library. Events are C# interface contracts (not concrete classes) which MassTransit serializes over RabbitMQ. This ensures a clean contract boundary and allows independent versioning of producers and consumers.

Event / Command	Producer	Consumer(s)	Purpose
UserRegisteredEvent	Identity	Admin	Create staff profile on registration
CategoryCreatedEvent	Admin	Catalog	Sync categories across services
CheckoutInitiatedEvent	Orders	Orders Saga	Start checkout saga, trigger payment
ProcessPaymentCommand	Orders Saga	Payment	Request payment processing
PaymentProcessedEvent	Payment	Orders Saga	Continue saga — deduct stock
SagaDeductStockCommand	Orders Saga	Catalog	Deduct purchased inventory
OrderPlacedEvent	Orders Saga	Admin, Notification	Sync order to admin, notify user
ReturnInitiatedEvent	Returns	Returns Saga	Start return saga
OrderReturnedEvent	Returns Saga	Catalog, Admin	Restore stock, sync admin
PaymentRefundedEvent	Payment	Returns Saga	Complete return flow

3. Microservices Reference

3.1 Identity Service

Port 5001

Purpose	Central authentication hub. Manages users, roles, permissions, and stores. Issues JWT access tokens at
Controllers	AuthController — login, register, verify email, OTP flows, Google OAuth, refresh token, password reset.
Key Flows	Registration → Email OTP verification → Login → OTP challenge (2FA) → JWT issued. Google OAuth sl
Saga Integration	Publishes UserRegisteredEvent upon successful registration. Participates in OnboardingStateMachine s
Models	User, Role, Permission, RolePermission, UserStoreRole, Store — all scoped to tenant (Store).

3.2 Catalog Service

Port 5002

Purpose	Product and category management with per-store stock tracking.
Controllers	ProductsController (CRUD + restore soft-deleted), CategoriesController, SyncController.
Consumers	CategoryConsumer (sync categories from Admin), SagaDeductStockConsumer (reduce stock on order),
Soft Delete	Products support soft-delete (IsDeleted flag) with restore capability to prevent orphaned order history.

3.3 Orders Service

Port 5003

Purpose	Manages the full order lifecycle from cart creation to finalization.
Controllers	BillsController (cart CRUD, finalize, hold, void), CustomersController.
Bill Statuses	Draft → Held → Finalized (or Voided).
Checkout Saga	On finalize, publishes CheckoutInitiatedEvent. The CheckoutStateMachine coordinates Payment → Stoc

3.4 Payment Service

Port 5005

Purpose	Processes payments and refunds for all transaction types.
---------	---

Consumers	Consumes ProcessPaymentCommand and RefundPaymentCommand from the message bus.
Payment Modes	Cash, Card, UPI — stored per transaction with status tracking (Pending, Completed, Failed, Refunded).
Events Published	PaymentProcessedEvent (success), PaymentFailedEvent (failure), PaymentRefundedEvent (refund).

3.5 Returns Service

Port 5006

Purpose	Handles merchandise returns with approval gates and refund coordination.
Return Saga	ReturnInitiatedEvent triggers the return saga. Saga coordinates: stock restoration (Catalog), refund proce
Events	Publishes ReturnApprovedEvent and ReturnRejectedEvent; consumes FinalizeReturnCommand.

3.6 Admin Service

Port 5004

Purpose	Back-office management including stores, staff, inventory, categories, reports, and dashboard KPIs.
Controllers	DashboardController, InventoryController, ReportsController, StoresController, StaffController, Categorie
Dashboard	Aggregates real-time KPIs: daily revenue, transaction count, top products, low-stock alerts.
Onboarding Saga	OnboardingStateMachine ensures atomic user onboarding across Identity and Admin services.
Multi-Tenancy	ITenantProvider extracts StoreId from JWT claims; all queries are scoped to the active store.

3.7 Notification Service

Port 5176

Purpose	Real-time push notifications to connected Angular clients via SignalR.
Hub	NotificationHub — clients subscribe to store-scoped notification groups.
Consumers	Listens for OrderPlacedEvent, DashboardEventsConsumer and others to push live updates.

3.8 AI Service

Port 5007

Purpose	Provides AI-powered chat assistance via the Claude API.
Controller	ChatController — single endpoint accepting user messages and returning AI responses.

Integration	Stateless HTTP proxy to Anthropic Claude API; no RabbitMQ dependency.
-------------	---

3.9 API Gateway

Port 5000

Purpose	Single entry point for all client requests. Routes, load-balances, and applies cross-cutting concerns.
Technology	Ocelot library configured via ocelot.json (local) and ocelot.docker.json (containerized).
Circuit Breaker	Each route configured with 3 exceptions allowed before breaking, 10-second break duration, 5-second tir
Auth Passthrough	JWT tokens are forwarded to downstream services; each service independently validates claims.

4. Authentication & Authorization

4.1 Authentication Flow

	User submits email + password to POST /gateway/auth/login
Step 2	IdentityService verifies credentials (BCrypt hash comparison)
	If valid, a 6-digit OTP is emailed to the registered address
Step 4	User submits OTP to POST /gateway/auth/verify-login-otp
	IdentityService issues: Access Token (JWT, 15 min) + Refresh Token (HttpOnly cookie, 7 days)
Step 6	Angular stores Access Token in memory; Refresh Token in HttpOnly cookie (XSS-safe)
	Auth interceptor automatically refreshes access token on 401 responses
Alt A	Google OAuth: POST /gateway/auth/google-login with Firebase ID token; same JWT output
	Password Reset: send-otp → reset-password with OTP code

4.2 JWT Token Strategy

Access tokens are short-lived (15 minutes) JWTs containing `userId`, `email`, `storeId`, `roleId`, `roleName`, and a permission claims array. Refresh tokens (7 days) are stored as HttpOnly cookies and exchanged for new access tokens via POST /gateway/auth/refresh. This two-token strategy limits exposure while maintaining session continuity.

4.3 Role-Based Access Control

Role	Scope	Key Capabilities
SuperAdmin	System-wide	Full access (all:all permission), cross-store management
Admin	All stores	Store management, reports, staff administration, inventory
StoreManager	Single store	Inventory adjustments, view reports, approve returns
Cashier	Single store	Create/finalize orders, process payments, initiate returns

4.4 Permission System

Permissions use a resource:action notation and are dynamically evaluated by the custom PermissionPolicyProvider and PermissionHandler. The SuperAdmin role is granted the all:all wildcard which bypasses all individual permission checks.

Permission Code	Description	Roles
orders:create	Create new bills/carts	Cashier, StoreManager
orders:finalize	Finalize a bill to an order	Cashier, StoreManager
orders:void	Void an existing bill	StoreManager, Admin
orders:view_all	View orders across all cashiers	StoreManager, Admin
returns:initiate	Start a merchandise return	Cashier, StoreManager
returns:approve	Approve/reject return requests	StoreManager, Admin
catalog:manage	Create/edit/restore products	StoreManager, Admin
catalog:delete	Soft-delete products	Admin, SuperAdmin
admin:inventory:adjust	Manual stock adjustments	StoreManager, Admin
admin:stores:manage	Create and edit stores	Admin, SuperAdmin
admin:reports:view	Access sales reports	StoreManager, Admin
all:all	Unrestricted system access	SuperAdmin

5. Data Model & Contracts

5.1 Domain Models

User (IdentityService)

- Id, Email, PasswordHash, FullName, IsEmailVerified, IsApproved, GoogleId
- Relations: UserStoreRole (many-to-many with Role per Store)

Store

- Id, StoreCode, Name, Location, IsActive
- Created by Admin; referenced by Identity, Admin, Orders, Catalog

Product (CatalogService)

- Id, Name, Price, StockQuantity, CategoryId, StoreId, IsDeleted
- Soft-deleted to preserve order history integrity

Bill (OrdersService)

- Id, BillNumber, Date, StoreId, CashierId, CustomerMobile, TotalAmount, TaxAmount, Status
- Status: Draft | Held | Finalized | Voided

BillItem (OrdersService)

- Id, BillId, ProductId, ProductName, UnitPrice, Quantity, LineTotal

Payment (PaymentService)

- Id, OrderId, Amount, PaymentMode, Status, CreatedAt
- Status: Pending | Completed | Failed | Refunded

Return (ReturnsService)

- Id, OrderId, StoreId, TotalRefund, Status, CustomerMobile
- Status: Pending | Approved | Rejected | Completed

StaffMember (AdminService)

- UserId, Email, FullName, RoleName, StoreId — mirrored from Identity via events

InventoryAdjustment (AdminService)

- Id, ProductId, StoreId, AdjustmentQty, Reason, CreatedAt

5.2 Message Contracts

All contracts are defined as C# interfaces in the RetailPOS.Contracts shared library. Using interfaces rather than classes allows MassTransit to handle polymorphic deserialization and enables contract evolution without breaking existing consumers.

5.3 Saga Patterns

Saga	Service	States	Timeout	Compensation
CheckoutStateMachine	Orders	Initial → PaymentProcessing → StockDeducting → Completed / Failed / TimedOut	5 min	PaymentRefunded event triggers refund; stock not deducted if payment fails
OnboardingStateMachine	Admin	Initial → WaitingForProfile → Completed	None	Manual retry via DataRepairController
ReturnStateMachine	Returns	Initial → StockRestoring → RefundProcessing → Completed / Rejected	None	Rejection event stops saga; stock already restored before refund

6. API Reference

All endpoints are accessed through the API Gateway at `http://localhost:5000/gateway/...` The Gateway strips the `/gateway` prefix before forwarding to downstream services. All protected routes require Bearer in the Authorization header.

6.2 Auth Endpoints

Method	Endpoint	Auth	Description
POST	/gateway/auth/login	Public	Submit email+password; returns OTP sent confirmation
POST	/gateway/auth/verify-login-otp	Public	Submit OTP; returns JWT access token
POST	/gateway/auth/register	Public	Register new user with store and role
POST	/gateway/auth/verify-email	Public	Verify email OTP after registration
POST	/gateway/auth/resend-verification	Public	Resend email verification OTP
POST	/gateway/auth/resend-login-otp	Public	Resend 2FA login OTP
POST	/gateway/auth/google-login	Public	Google OAuth sign-in with Firebase ID token
POST	/gateway/auth/send-otp	Public	Send password reset OTP
POST	/gateway/auth/reset-password	Public	Reset password with OTP
POST	/gateway/auth/refresh	Cookie	Refresh access token using HttpOnly cookie
GET	/gateway/auth/users	JWT	List all users (Admin+)
GET	/gateway/auth/stores	Public	List all stores (for registration form)

6.3 Catalog Endpoints

Method	Endpoint	Auth	Description
GET	/gateway/catalog/products	catalog:view	List active products for current store
GET	/gateway/catalog/products/all	catalog:view	List all products including soft-deleted
POST	/gateway/catalog/products	catalog:manage	Create a new product
PUT	/gateway/catalog/products/{id}	catalog:manage	Update an existing product
DELETE	/gateway/catalog/products/{id}	catalog:delete	Soft-delete a product
POST	/gateway/catalog/products/{id}/restore	catalog:manage	Restore a soft-deleted product
GET	/gateway/catalog/categories	catalog:categories:view	List categories for current store
POST	/gateway/catalog/categories	catalog:categories:edit	Create a category
PUT	/gateway/catalog/categories/{id}	catalog:categories:edit	Update a category

6.4 Orders Endpoints

Method	Endpoint	Auth	Description
GET	/gateway/orders/bills	orders:view_all	List all bills for the store
GET	/gateway/orders/bills/{id}	orders:view	Get bill details by ID
POST	/gateway/orders/bills	orders:create	Create or update a bill/cart
POST	/gateway/orders/bills/cart/items	orders:create	Add/update items in active cart
POST	/gateway/orders/bills/{id}/finalize	orders:finalize	Finalize bill → triggers checkout saga
POST	/gateway/orders/bills/{id}/hold	orders:hold	Place bill on hold
POST	/gateway/orders/bills/{id}/void	orders:void	Void a bill (Manager action)

6.5 Admin Endpoints

Method	Endpoint	Auth	Description
GET	/gateway/admin/dashboard	admin:reports:view	KPI summary for current store
GET	/gateway/admin/reports/sales	admin:reports:view	Sales report with filters
GET	/gateway/admin/inventory	admin:stores:view	Current inventory levels
POST	/gateway/admin/inventory/adjust	admin:inventory:adjust	Manual stock adjustment with reason
GET	/gateway/admin/stores	admin:stores:view	List all stores
POST	/gateway/admin/stores	admin:stores:manage	Create a new store
PUT	/gateway/admin/stores/{id}	admin:stores:manage	Update store details
GET	/gateway/admin/staff	admin:stores:view	List staff members for store
POST	/gateway/admin/data-repair	all:all	Trigger data repair / saga retry

7. Frontend Architecture

7.1 Angular Application Structure

The Angular 18 application uses standalone components and functional routing with lazy-loading per feature module. The application is structured into three primary zones: Auth (login/signup), POS (cashier workflow), and Admin (management).

core/

- `auth.guard.ts` — redirects unauthenticated users
- `auth.interceptor.ts` — attaches Bearer token to all requests
- `error.interceptor.ts` — handles 401 (auto-refresh) and 500 errors
- `signalr.service.ts` — manages SignalR hub connection for real-time notifications
- `theme.service.ts` — dark/light mode toggle

features/auth/

- `login/` — email+password form, OTP verification step
- `signup/` — registration with store and role selection
- `otp-verification/` — OTP input component
- `pending-approval/` — shown while admin approves new user

features/pos/

- `billing/` — main cashier screen: product search, cart, totals
- `payment/` — payment mode selection and confirmation
- `payment-dialog/` — modal dialog for payment flow
- `returns/` — return initiation form with order lookup

features/admin/

- `dashboard/` — KPI cards, charts, low-stock alerts
- `inventory/` — stock levels table with adjustment modal
- `categories/` — category CRUD
- `products/` — product management grid
- `staff/` — staff list and role management
- `stores/` — store management
- `transactions/` — order history with filters
- `returns/` — pending returns approval queue

shared/components/

- `ai-chatbot/` — floating AI assistant panel (Claude integration)

- navbar/ — top navigation with user info and notifications
- sidebar/ — role-aware navigation menu

7.2 Services & State

Service	Purpose
auth.service.ts	Token management, login state, role/permission helpers
auth-api.service.ts	HTTP calls to Identity service endpoints
catalog-api.service.ts	Product and category API calls
order-api.service.ts	Bill CRUD and finalize API calls
payment-api.service.ts	Payment submission and status polling
admin-api.service.ts	Dashboard, inventory, reports, store management calls
ai-api.service.ts	AI chatbot message submission
signalr.service.ts	Real-time notification hub connection
notification.service.ts	Toast notification display management

8. Deployment

8.1 Docker Compose Setup

The entire stack is orchestrated with Docker Compose. Infrastructure containers (db, rabbitmq, seq, redis) start first with health checks. Application services depend on the database and message broker being healthy before starting. All services share the retailpos-network bridge network.

To start the full stack:

```
docker compose up --build
```

8.2 Port Allocation

Service	Host Port	Container Port	Notes
API Gateway	5000	8080	Primary entry point for all clients
Identity Service	5001	8080	Auth and user management
Catalog Service	5002	8080	Products and categories
Orders Service	5003	8080	Bills and checkout
Admin Service	5004	8080	Back-office management
Payment Service	5005	8080	Payment processing
Returns Service	5006	8080	Return management
AI Service	5007	8080	AI chatbot proxy
Notification Service	5176	8080	SignalR hub
Angular UI	4200	4200	Frontend development server
SQL Server	1433	1433	Database server
RabbitMQ AMQP	5672	5672	Message broker
RabbitMQ Management	15672	15672	RabbitMQ web UI
Seq Ingestion	5341	5341	Log ingestion HTTP endpoint
Seq Dashboard	8081	80	Log viewer web UI
Redis	6379	6379	In-memory cache

8.3 Environment Variables

Secrets and configuration are stored in a .env file (gitignored) and injected via env_file in Docker Compose. Key environment variables include:

ConnectionStrings__DefaultConnection	MS SQL Server connection string
JwtSettings__Secret	JWT signing secret (minimum 32 characters)
JwtSettings__Issuer	Token issuer identifier
JwtSettings__Audience	Token audience identifier
RabbitMQ__Host	RabbitMQ hostname (rabbitmq in Docker)
RabbitMQ__Username	RabbitMQ credentials
RabbitMQ__Password	RabbitMQ credentials
Email__Host	SMTP server hostname
Email__Port	SMTP port (typically 587)
Email__Username	SMTP sender address
Email__Password	SMTP app password
Google__ClientId	Google OAuth client ID (Firebase)
Anthropic__ApiKey	Claude API key for AI service
Seq__ServerUrl	Seq log ingestion URL
Redis__ConnectionString	Redis connection string

9. Testing Strategy

9.1 Unit Tests

The project includes xUnit unit test projects for all major services. Tests use the Moq library to mock dependencies and follow the Arrange-Act-Assert pattern. Each service test project mirrors the structure of its corresponding service.

<code>IdentityService.Tests</code>	AuthService — login flows, OTP generation, JWT issuance, Google OAuth
<code>CatalogService.Tests</code>	ProductService — CRUD operations, stock deduction, soft-delete/restore
<code>OrdersService.Tests</code>	BillService — cart creation, finalization, hold/void logic
<code>PaymentService.Tests</code>	PaymentService — payment processing, refund handling, status transitions
<code>ReturnsService.Tests</code>	ReturnService — return initiation, approval/rejection flows
<code>NotificationService.Tests</code>	NotificationService — event consumption, hub message dispatch
<code>AdminService.Tests</code>	DashboardService, InventoryService, ReportService

9.2 Running Tests

```
dotnet test # Run all tests
dotnet test Tests/OrdersService.Tests # Run specific service tests
```

9.3 Test Coverage Areas

- Authentication: Login success/failure, OTP expiry, token refresh, Google OAuth edge cases
- Authorization: Permission boundary enforcement, role escalation prevention
- Order Lifecycle: Cart creation, item updates, finalization, hold/void transitions
- Inventory: Stock deduction on order, restoration on return, manual adjustment validation
- Payment: Processing success/failure, refund amount validation, status transitions
- Saga Compensation: Payment failure triggers refund; stock not deducted on payment failure
- Multi-Tenancy: Store isolation — users cannot access data from other stores